In our Amazon S3, we have two folders containing the application code:

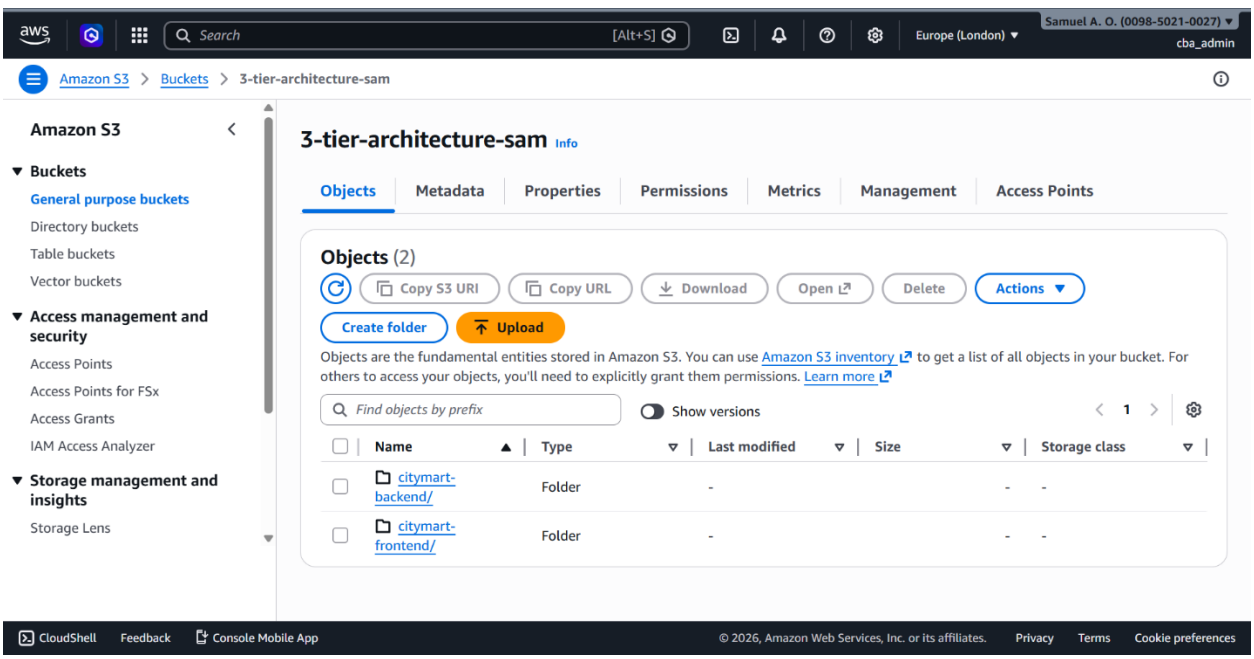
```
3-tier-architecture-sam bucket
│
├── citymart-frontend/
└── citymart-backend/
```

1. CityMart Application Deployment Structure

1. Application Code Storage in S3

The CityMart application code is stored in the S3 bucket:

```
3-tier-architecture-sam
```




```
3-tier-architecture-sam
│
├── citymart-frontend/
│
└── citymart-backend/
```

Each Auto Scaling Group pulls **only the code it needs**.

Tier	*Folder Pulled*
Frontend EC2	citymart-frontend
Backend EC2	citymart-backend

2. Frontend Application Folder (citymart-frontend)

This folder contains the **presentation layer of the application**, which is served by **Nginx running on the frontend EC2 instances**.

Typical structure:

```
The `citymart-frontend/` code directory contains:
citymart-frontend/
│
├── index.html
├── products.html
├── product.html
├── checkout.html
├── order-success.html
│
└── assets/
├── css/
│   └── styles.css
│
├── js/
│   ├── config.js
│   ├── api.js
│   ├── products.js
│   ├── product.js
│   └── checkout.js
│
└── img/
├── logo.png
├── rice.png
├── oil.png
├── soap.png
├── detergent.png
├── product-default.png
└── placeholder-product.png
```

These files are copied to the frontend server directory:

```
/var/www/html
```

and served by **Nginx**.

3. Frontend Application Functionality

The frontend provides the user interface for the CityMart online store.

Key pages:

```
index.html
products.html
```

```
product.html
checkout.html
order-success.html
```

These allow users to:

- browse products
- view product details
- submit order requests

The frontend uses JavaScript to call backend APIs.

Example calls:

```
GET /products
GET /product?id=1
POST /order
```

4. Backend Application Folder (citymart-backend)

This folder contains the **business logic and API implementation** of the application.

Typical structure:

```
The `citymart-backend/` code directory contains:
citymart-backend
│
├── apache
│   └── citymart.conf
│
├── api
│   ├── health.php
│   ├── order.php
│   ├── product.php
│   └── products.php
│
├── config
│   ├── app.php
│   └── database.php
│
├── public
│   └── index.php
│
├── scripts
│   ├── migrate.sql
│   ├── seed.sql
│   └── reset_db.sh
│
└── src
├── CustomersService.php
```

```
├── Database.php
├── Db.php
├── OrdersService.php
├── ProductsService.php
├── Response.php
└── Validator.php
```

These files are deployed to:

`/var/www/html/backend`

on the backend EC2 servers.

5. Backend API Endpoints

The backend exposes a small REST-style API used by the frontend.

Health check endpoint:

`GET /api/health`

Product catalog:

`GET /api/products`

Product details:

`GET /api/product?id=1`

Order submission:

`POST /api/order`

These APIs interact with the **CityMart MySQL database**.

6. Database Layer

The backend connects to the RDS instance:

`citymart-database`

Endpoint:

`citymart-database.cdgowmasatge.eu-west-2.rds.amazonaws.com`

Database name:

`citymart`

Main tables:

```
products
customers
orders
order_items
```

The database configuration is defined in:

```
citymart-backend/config/database.php
```

7. How the Application Code Is Pulled to Servers

Both frontend and backend EC2 instances download their code from S3 at startup.

This is possible because the EC2 instances use the IAM role:

```
3-tier-architecture-sam
```

which has permissions:

```
s3:ListBucket
s3:GetObject
```

for the bucket:

```
3-tier-architecture-sam
```

Frontend deployment process

When a frontend EC2 instance starts:

```
EC2 startup
↓
User data script runs
↓
Download citymart-frontend from S3
↓
Copy files to /var/www/html
↓
Start Nginx
```

Backend deployment process

When a backend EC2 instance starts:

```
EC2 startup
↓
User data script runs
↓
Download citymart-backend from S3
↓
Copy files to /var/www/html/backend
↓
Start Apache + PHP
```

8. How the Full Application Works

The final request flow through the architecture is:

```
User
↓
CloudFront
↓
Route53
↓
Public ALB
↓
Frontend EC2 (Nginx)
↓
Internal ALB
↓
Backend EC2 (Apache + PHP)
↓
RDS MySQL (citymart database)
```

The frontend sends API requests to:

```
/api/*
```

Nginx then forwards these requests to the **Internal Application Load Balancer**, which distributes them to backend servers.

9. Why This Structure Is Better

Separating the application into two S3 folders provides several operational advantages.

Independent scaling

Frontend and backend servers can scale independently.

Faster deployments

Instances download **only the files they need**.

Cleaner architecture

Each tier has its own code base.

Better Auto Scaling support

New instances can start quickly by pulling their required code from S3.

Our CityMart architecture stores application code in S3 as:

```
3-tier-architecture-sam
│
├── citymart-frontend
└── citymart-backend
```

Frontend EC2 instances pull **citymart-frontend**, while backend EC2 instances pull **citymart-backend**, enabling clean separation of the presentation layer and application logic layer in the three-tier architecture.